

Binary Search

Sean

August 23, 2026



Outline

- Why binary search works
- Binary search on sorted arrays
- First true / last true
- Binary search on the answer



Introduction

- Binary search repeatedly cuts the search space in half

Introduction

- Binary search repeatedly cuts the search space in half
- It works when the answer changes from impossible to possible at a boundary



Introduction

- Binary search repeatedly cuts the search space in half
- It works when the answer changes from impossible to possible at a boundary
- In this lecture, we focus on **discrete** ordered spaces like indices, sorted values, and integer answers



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



Why Binary Search Works

When to Think Binary Search

- We need a fast way to find a position, value, or threshold



When to Think Binary Search

- We need a fast way to find a position, value, or threshold
- The valid answers form an ordered range



When to Think Binary Search

- We need a fast way to find a position, value, or threshold
- The valid answers form an ordered range
- We can tell whether the answer is to the left or right of a midpoint



When to Think Binary Search

- We need a fast way to find a position, value, or threshold
- The valid answers form an ordered range
- We can tell whether the answer is to the left or right of a midpoint
- Each check removes half of the remaining search space



Complexity

- One step reduces the interval size by about half



Complexity

- One step reduces the interval size by about half
- After k steps, only about $\frac{n}{2^k}$ candidates remain



Complexity

- One step reduces the interval size by about half
- After k steps, only about $\frac{n}{2^k}$ candidates remain
- That gives $\mathcal{O}(\log n)$ checks instead of $\mathcal{O}(n)$



Complexity

- One step reduces the interval size by about half
- After k steps, only about $\frac{n}{2^k}$ candidates remain
- That gives $\mathcal{O}(\log n)$ checks instead of $\mathcal{O}(n)$
- Binary search is fast, but only when the search condition is correct



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



Binary Search on Sorted Arrays

Sorted Array Search

- Keep a search interval $[l, r]$

Sorted Array Search

- Keep a search interval $[l, r]$
- Look at the middle position

Sorted Array Search

- Keep a search interval $[l, r]$
- Look at the middle position
- If the middle value is too small, discard the left half



Sorted Array Search

- Keep a search interval $[l, r]$
- Look at the middle position
- If the middle value is too small, discard the left half
- If the middle value is large enough, discard the right half or record a candidate



Exact Match Template

```
1  int l = 0, r = n - 1;
2  while (l <= r) {
3      int mid = l + (r - l) / 2;
4      if (a[mid] == x) return mid;
5      if (a[mid] < x) l = mid + 1;
6      else r = mid - 1;
7  }
8  return -1;
```



Trace on 11 Values

Find 25 in the sorted array below.

$l = 0$ ↓					$mid = 5$ ↓					$r = 10$ ↓
2	4	7	9	13	18	21	25	31	40	44
0	1	2	3	4	5	6	7	8	9	10

$a[mid] = 18 < 25$, so set $l = 6$



Trace on 11 Values

Find 25 in the sorted array below.

						$l = 6$	$mid = 8$	$r = 10$		
2	4	7	9	13	18	21	25	31	40	44
0	1	2	3	4	5	6	7	8	9	10

$a[mid] = 31 > 25$, so set $r = 7$



Trace on 11 Values

Find 25 in the sorted array below.

$l = mid = 6$										
						$\downarrow$	$r = 7$			
2	4	7	9	13	18	21	25	31	40	44
0	1	2	3	4	5	6	7	8	9	10

$a[mid] = 21 < 25$, so set $l = 7$

Trace on 11 Values

Find 25 in the sorted array below.

$l = mid = r = 7$

2	4	7	9	13	18	21	25	31	40	44
0	1	2	3	4	5	6	7	8	9	10

$a[mid] = 25$, so we found the answer

Common Invariants

- An **invariant** is a fact that stays true every time the loop runs



Common Invariants

- An **invariant** is a fact that stays true every time the loop runs
- For example: “if the answer exists, it is still inside the current search interval”



Common Invariants

- An **invariant** is a fact that stays true every time the loop runs
- For example: “if the answer exists, it is still inside the current search interval”
- Make sure every update keeps that fact true and also shrinks the interval



Common Invariants

- An **invariant** is a fact that stays true every time the loop runs
- For example: “if the answer exists, it is still inside the current search interval”
- Make sure every update keeps that fact true and also shrinks the interval
- Most bugs come from breaking the invariant or mixing interval conventions



Template Conventions

- Competitive programmers use several valid binary search templates



Template Conventions

- Competitive programmers use several valid binary search templates
- Some use closed intervals like $[l, r]$, and others use **half-open intervals** like $[l, r)$



Template Conventions

- Competitive programmers use several valid binary search templates
- Some use closed intervals like $[l, r]$, and others use **half-open intervals** like $[l, r)$
- Some exact-search templates use `while (l <= r)`, while many boundary-search templates use `while (l < r)`



Template Conventions

- Competitive programmers use several valid binary search templates
- Some use closed intervals like $[l, r]$, and others use **half-open intervals** like $[l, r)$
- Some exact-search templates use `while (l <= r)`, while many boundary-search templates use `while (l < r)`
- You do not need to memorize every version; pick one style, understand its invariant, and use it consistently



Midpoint Choices

- Many people write the midpoint as $(1 + r) / 2$



Midpoint Choices

- Many people write the midpoint as $(1 + r) / 2$
- $1 + (r - 1) / 2$ is safer because it avoids [overflow](#)



Midpoint Choices

- Many people write the midpoint as $(1 + r) / 2$
- $1 + (r - 1) / 2$ is safer because it avoids **overflow**
- Some templates use the lower midpoint and others use the upper midpoint



Midpoint Choices

- Many people write the midpoint as $(l + r) / 2$
- $l + (r - l) / 2$ is safer because it avoids **overflow**
- Some templates use the lower midpoint and others use the upper midpoint
- The midpoint formula must match your update rules, or the loop can get stuck



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



First True / Last True

Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true



Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true
- Think of a boolean array of the form false, false, ..., true, true



Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true
- Think of a boolean array of the form false, false, ..., true, true
- Binary search finds the boundary between the two regions



Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true
- Think of a boolean array of the form false, false, ..., true, true
- Binary search finds the boundary between the two regions
- This is the pattern behind `lower_bound` and `upper_bound`



First True Template

```
1  int l = 0, r = n - 1, ans = n;
2  while (l <= r) {
3      int mid = l + (r - l) / 2;
4      if (good(mid)) {
5          ans = mid;
6          r = mid - 1;
7      } else {
8          l = mid + 1;
9      }
10 }
```



Typical Uses

- First element greater than or equal to x

Typical Uses

- First element greater than or equal to x
- First element strictly greater than x

Typical Uses

- First element greater than or equal to x
- First element strictly greater than x
- Last position where a predicate is true

Typical Uses

- First element greater than or equal to x
- First element strictly greater than x
- Last position where a predicate is true
- Counting values in a sorted range using two boundaries

lower_bound and upper_bound

```
1  int i = lower_bound(a.begin(), a.end(), x) - a.begin();  
2  int j = upper_bound(a.begin(), a.end(), x) - a.begin();
```

lower_bound and upper_bound

```
1  int i = lower_bound(a.begin(), a.end(), x) - a.begin();  
2  int j = upper_bound(a.begin(), a.end(), x) - a.begin();
```

- lower_bound finds the first position with value $\geq x$



lower_bound and upper_bound

```
1  int i = lower_bound(a.begin(), a.end(), x) - a.begin();  
2  int j = upper_bound(a.begin(), a.end(), x) - a.begin();
```

- lower_bound finds the first position with value $\geq x$
- upper_bound finds the first position with value $> x$



lower_bound and upper_bound

```
1  int i = lower_bound(a.begin(), a.end(), x) - a.begin();  
2  int j = upper_bound(a.begin(), a.end(), x) - a.begin();
```

- lower_bound finds the first position with value $\geq x$
- upper_bound finds the first position with value $> x$
- The number of copies of x is $j - i$



lower_bound and upper_bound

```
1  int i = lower_bound(a.begin(), a.end(), x) - a.begin();  
2  int j = upper_bound(a.begin(), a.end(), x) - a.begin();
```

- `lower_bound` finds the first position with value $\geq x$
- `upper_bound` finds the first position with value $> x$
- The number of copies of x is $j - i$
- After sorting, `unique` can remove consecutive duplicates



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



Binary Search on the Answer

Searching the Answer Space

- Sometimes the array is not the search space



Searching the Answer Space

- Sometimes the array is not the search space
- Instead, guess an answer and check whether it is feasible



Searching the Answer Space

- Sometimes the array is not the search space
- Instead, guess an answer and check whether it is feasible
- The check must be **monotonic**: once it becomes true, it stays true



Searching the Answer Space

- Sometimes the array is not the search space
- Instead, guess an answer and check whether it is feasible
- The check must be **monotonic**: once it becomes true, it stays true
- Then binary search finds the smallest or largest feasible answer



Example: Factory Machines

CSES: Factory Machines

Given machine times, find the smallest time needed to produce at least t products.

Example: Factory Machines

CSES: Factory Machines

Given machine times, find the smallest time needed to produce at least t products.

- Guess a time mid

Example: Factory Machines

CSES: Factory Machines

Given machine times, find the smallest time needed to produce at least t products.

- Guess a time mid
- Compute how many products all machines can make by time mid



Example: Factory Machines

CSES: Factory Machines

Given machine times, find the smallest time needed to produce at least t products.

- Guess a time mid
- Compute how many products all machines can make by time mid
- If that total is at least t , try a smaller time



Example: Factory Machines

CSES: Factory Machines

Given machine times, find the smallest time needed to produce at least t products.

- Guess a time mid
- Compute how many products all machines can make by time mid
- If that total is at least t , try a smaller time
- Otherwise, try a larger time



Common Mistakes

- Overflow in midpoint computation

Common Mistakes

- Overflow in midpoint computation
- Infinite loops from not shrinking the interval



Common Mistakes

- Overflow in midpoint computation
- Infinite loops from not shrinking the interval
- Using binary search on a non-monotonic check



Common Mistakes

- Overflow in midpoint computation
- Infinite loops from not shrinking the interval
- Using binary search on a non-monotonic check
- Forgetting edge cases where no answer exists or every value works



Takeaways

- Binary search needs order plus a clear invariant
- Exact search and boundary search are closely related
- `lower_bound` and `upper_bound` are standard tools
- Answer-space binary search depends on a monotonic check



5 Appendix

Appendix

Appendix: Invariant

- An invariant is a fact that stays true during the whole loop
- Example: “if the answer exists, it is still inside the current interval”
- Every update to l or r should keep that fact true



Appendix: Monotonic

- Monotonic means the check changes in only one direction
- Typical pattern: false, false, false, true, true, true
- That creates one clean boundary, which is why binary search works



Appendix: Discrete

- Discrete means the search space is made of separate countable choices
- Examples here: array indices, sorted values, and integer answers
- That is different from searching over all real numbers



Appendix: Half-Open Interval

- The interval $[l, r)$ includes l but does not include r
- So it contains positions $l, l + 1, \dots, r - 1$
- Many binary search templates use this style because the right endpoint stays exclusive



Appendix: Overflow

- Overflow means a value goes past the range its integer type can store
- If l and r are large, $l + r$ can overflow
- That is why $l + (r - l) / 2$ is safer than $(l + r) / 2$



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

